deti
universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

Brief Introduction to Operating Systems

Real-Time Embedded Systems (#41505)

Paulo Pedreiras, pbrp@ua.pt
Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2026/2027

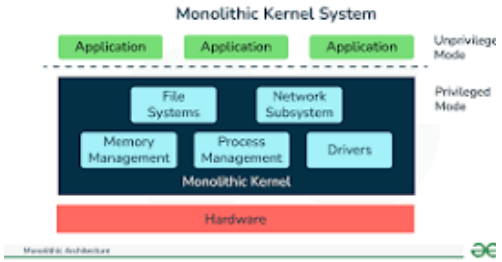


Outline

- To know general organization of a computer system
- To know the main components of multiprocessor computer system
- To know the general architecture of an Operating System (OS)
- To know the main resource types managed by OSs
- Understand modes and the transition from user mode to kernel mode
- Discuss how operating systems are used in various computing environments







Preliminary note

- This course has **students with different backgrounds**
- All students are supposed to have a **good knowledge of C programming**
- But some students had one or two courses on Computer Architecture, while others didn't!

  Significant impact in these preliminary topics



- Anyway, a basic knowledge of Operating Systems is essential, so:

  If you have a good background in C and computer architecture

  - Lucky you! Enjoy! Topics addressed today are very light ...

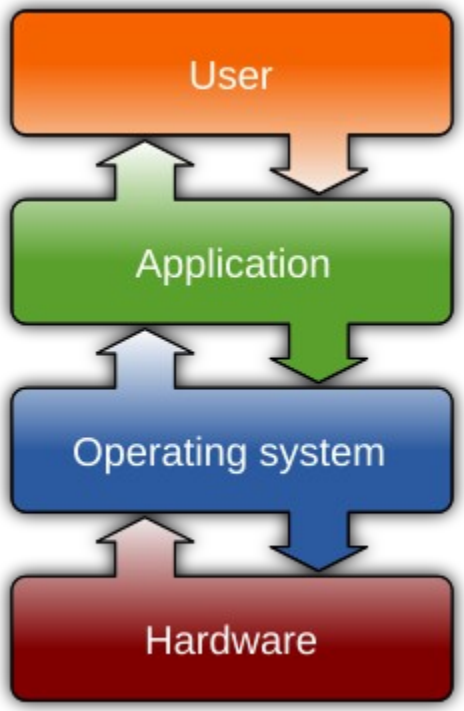
  If you do not have such background:

  - Read carefully the recommended chapters
  - If necessary contact me to book a session to clarify eventual doubts.

    Time in T class is very limited! I cannot go very deep ...

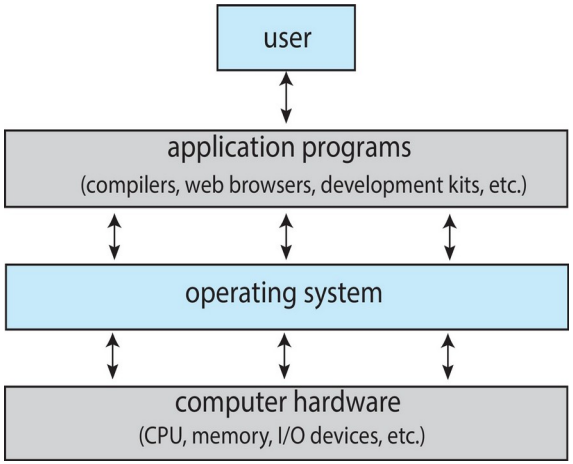
What is an Operating System?

- A program that acts as an intermediary between a user of a computer and the computer hardware
- Operating system main goals:
  - Execute user programs easily and efficiently
  - Make the computer system convenient to use
  - **Abstract the hardware**
  - Use the computer hardware in an efficient manner



Computer System Structure

- Computer systems can be divided into four components:
  - **Hardware** – provides basic computing resources
    - CPU, memory, I/O devices
  - **Operating System**
    - Controls and coordinates use of hardware among various applications and users
  - **Application programs** – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - **Users**
    - People, machines, other computers – the ones who benefit from the previous three components



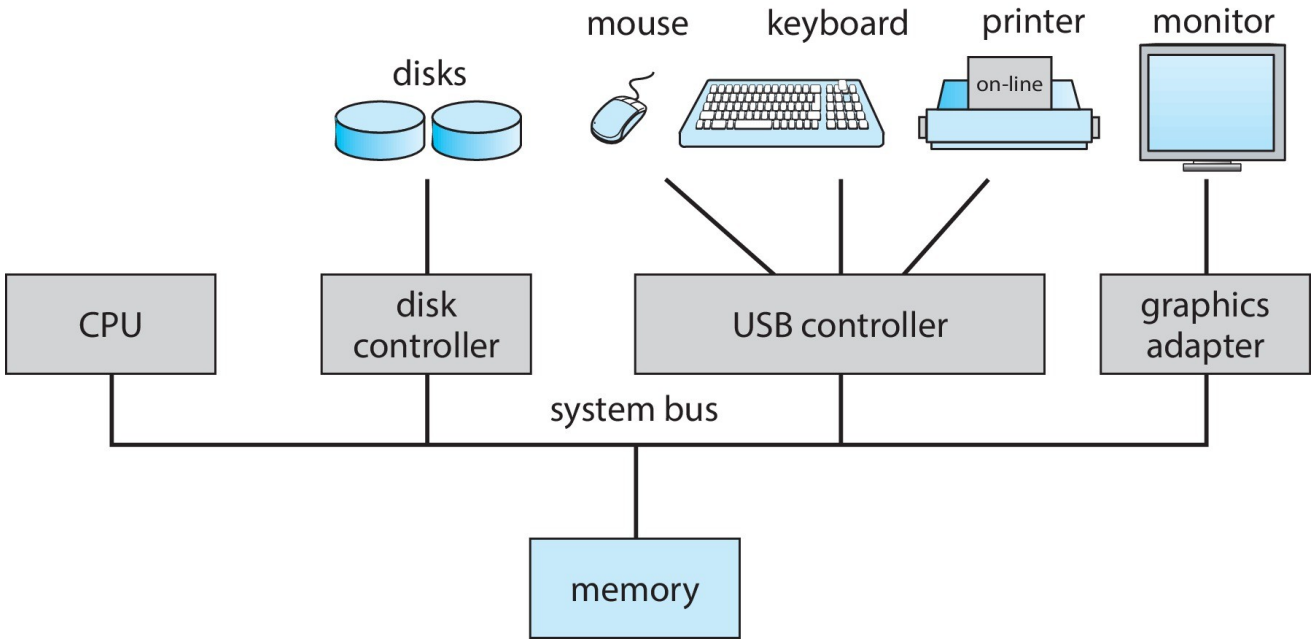
Operating System Definition

- **No universally accepted definition**
- "Everything a vendor ships when you order an operating system" is a good approximation
  - But varies wildly
- "The one program running at all times on the computer" is the **kernel,** part of the operating system
- Everything else is either
  - A ***system program*** (ships with the operating system, but not part of the kernel) , or
  - An ***application program***, all programs not associated with the operating system
- Today's OSes for general purpose and mobile computing also include ***middleware*** – a set of software frameworks that provide additional services to application developers such as databases, multimedia, graphics

Overview of Computer Systems Structure

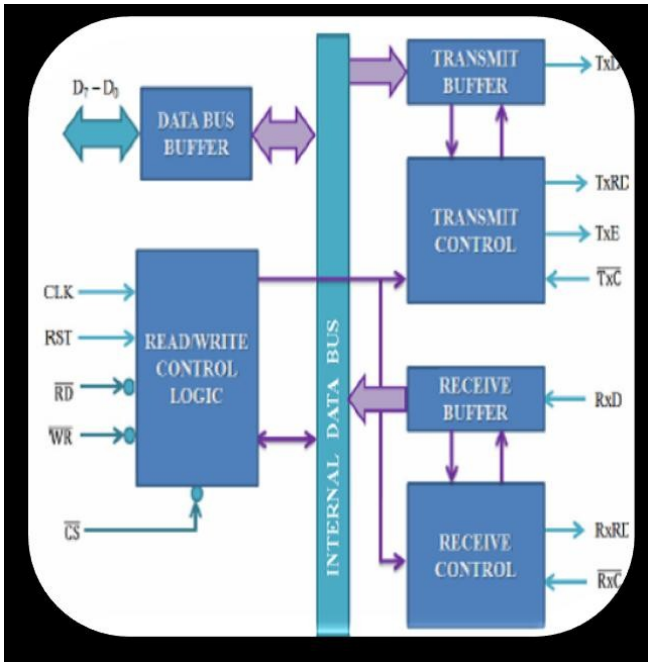
Computer System Organization

- Computer-system operation
  - One or more CPUs and a set of device controllers connect through a common **bus** (in reality several buses) providing access to shared memory
  - Concurrent execution of CPUs and devices competing for memory cycles



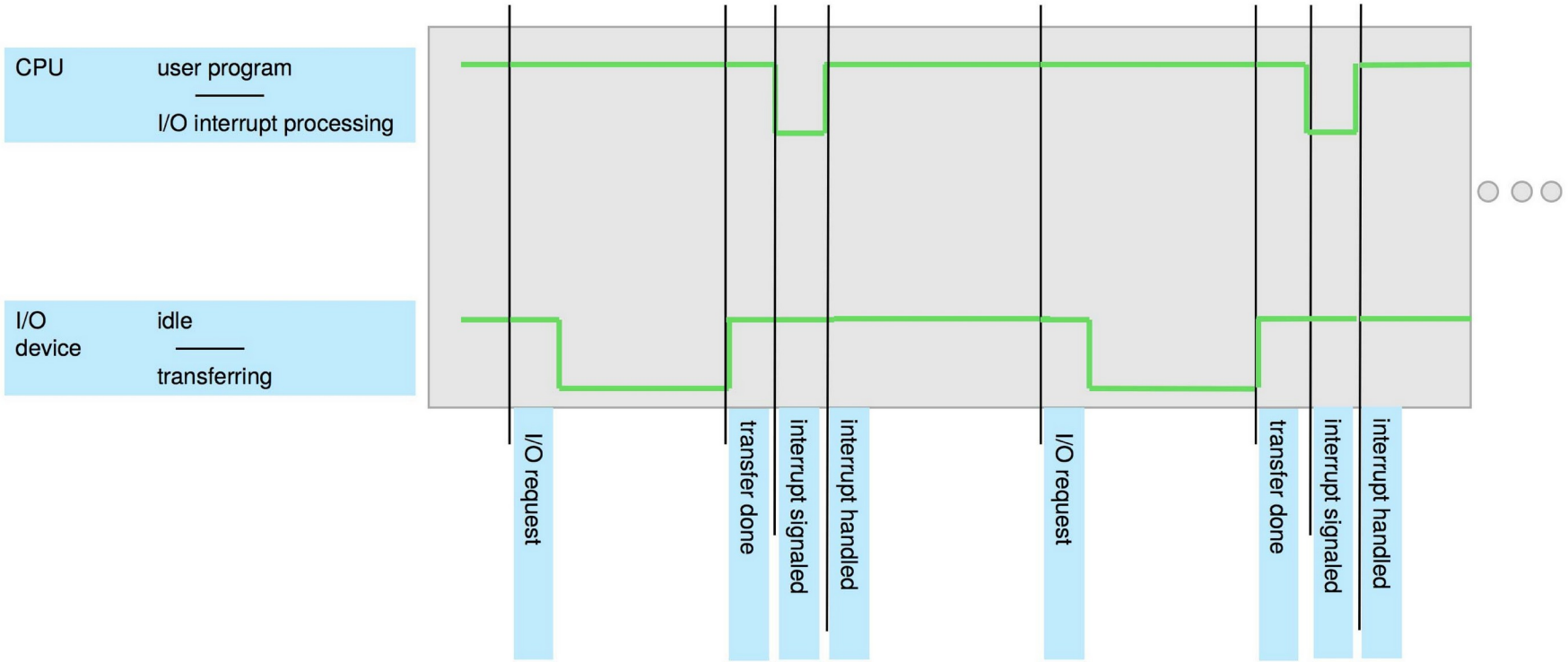
Computer-System Operation

- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- Each device controller type has an operating system **device driver** to manage it
- CPU moves data from/to main memory to/from local buffers
- I/O is from the device to local buffer of controller
- Device controllers inform the CPU of events (e.g. it has finished its operation) by causing **interrupts**



Simplified architecture of a USART

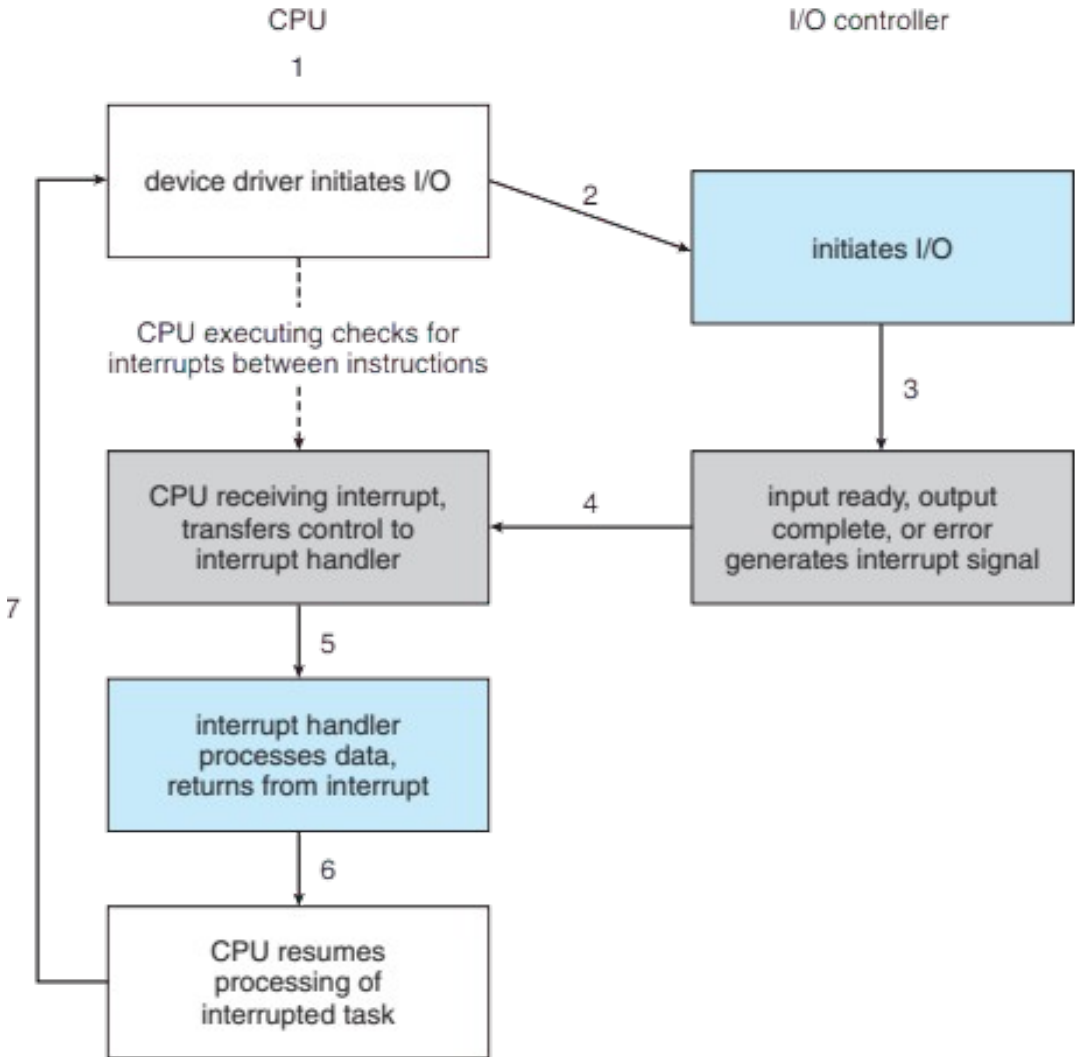
Interrupt Timeline



Interrupt Handling

- The operating system preserves the state of the CPU by storing the registers and the program counter – the so-called **context**
  - Interrupted activities can be resumed later by restoring the context
- Determines which type of interrupt has occurred
  - Separate segments of code determine what action should be taken for each type of interrupt
- Transfer control to the corresponding interrupt handler
  - Interrupt Service Routine (ISR) execution
- Resume normal operation. It can be e.g. resume the execution of the interrupted program, or trigger the execution of other program, e.g. in response to the interrupt (fast/slow handler)

Interrupt-drive I/O Cycle



I/O Structure

- Two methods of handling I/O interrupts
  - **Synchronous/Blocking:**
    - After I/O starts, control returns to user program only upon I/O completion
      - Wait instruction idles the CPU until the next interrupt (save energy) or there is a wait loop - polling
    - At most one I/O request is processed at a time, no simultaneous I/O processing
  - **Asynchronous/Non-blocking**
    - After I/O starts, control returns to user program without waiting for I/O completion
    - Eventually the program may have to wait for for I/O completion – system call for that (e.g. poll, select)
    - **Device-status table** holds an entry for each I/O device indicating its type, address, state and programs waiting
    - When the hardware fires an interrupt, it is executed an Interrupt Service Routine (ISR), the OS checks the device-status table to see which device it belongs to, marks the operation as complete, and wakes up the specific user program that was waiting for that data.

Storage Structure

Storage Structure

- Main memory – single large storage media that the CPU can **access directly**
  - Often called “RAM”
  - **Random access**
  - Typically **volatile**
  - Typically **random-access memory** in the form of **Dynamic Random-access Memory (DRAM)**

- Secondary storage – extension of main memory that provides large **nonvolatile** (i.e. permanent) storage capacity

Storage Structure (Cont.)

- **Hard Disk Drives** (**HDD**) – rigid metal or glass platters covered with magnetic recording material
  - Disk surface is logically divided into **tracks**, which are subdivided into **sectors**
  - The **disk controller** determines the logical interaction between the device and the computer

- **Non-volatile memory** (**NVM**) devices– faster than hard disks, nonvolatile
  - Various technologies
  - Becoming dominant as capacity and performance increases, price drops
  - **Often the only option in embedded hardware!**

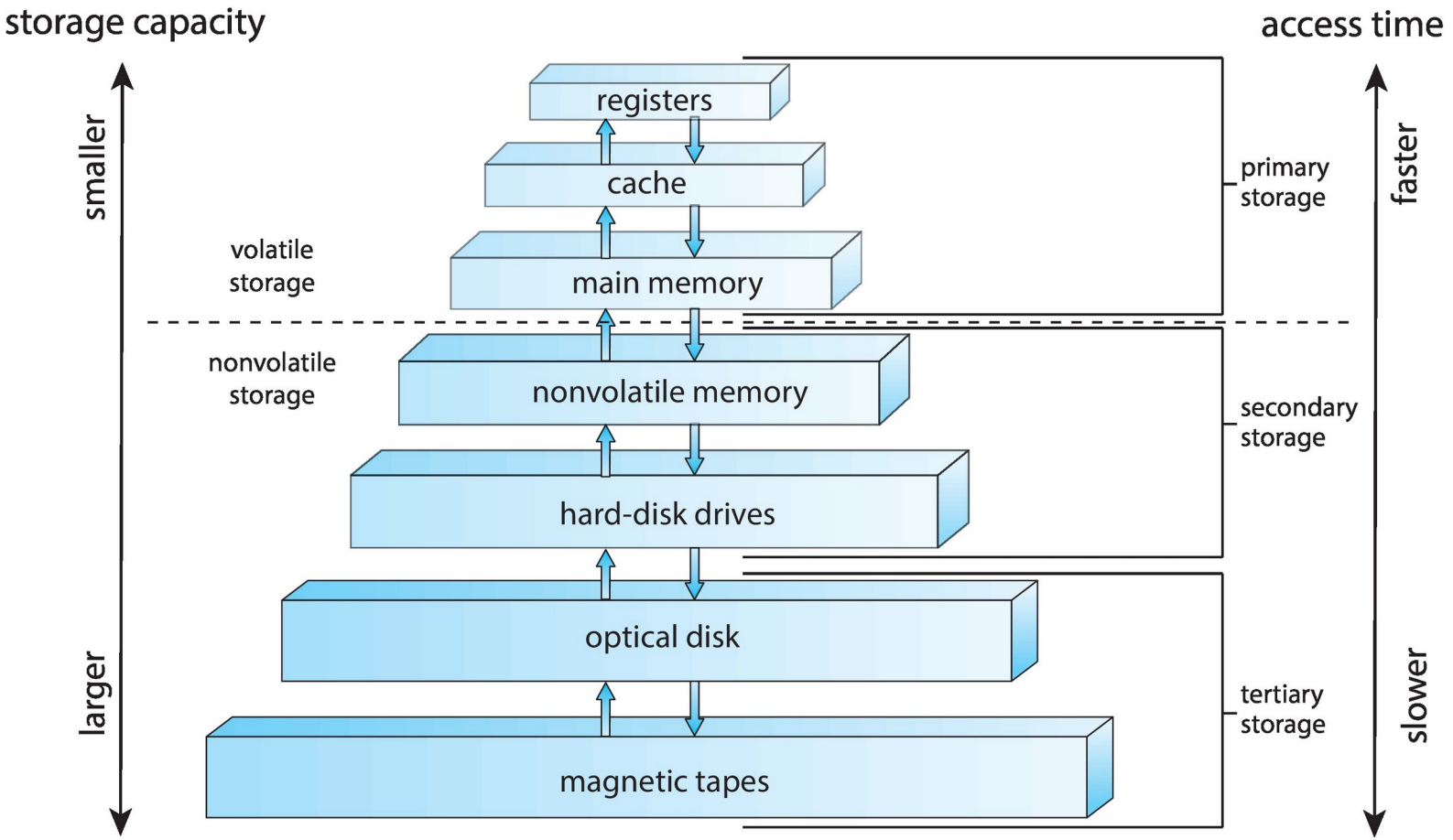
Storage Hierarchy

- Storage systems organized in hierarchy
  - Speed
  - Cost
  - Volatility

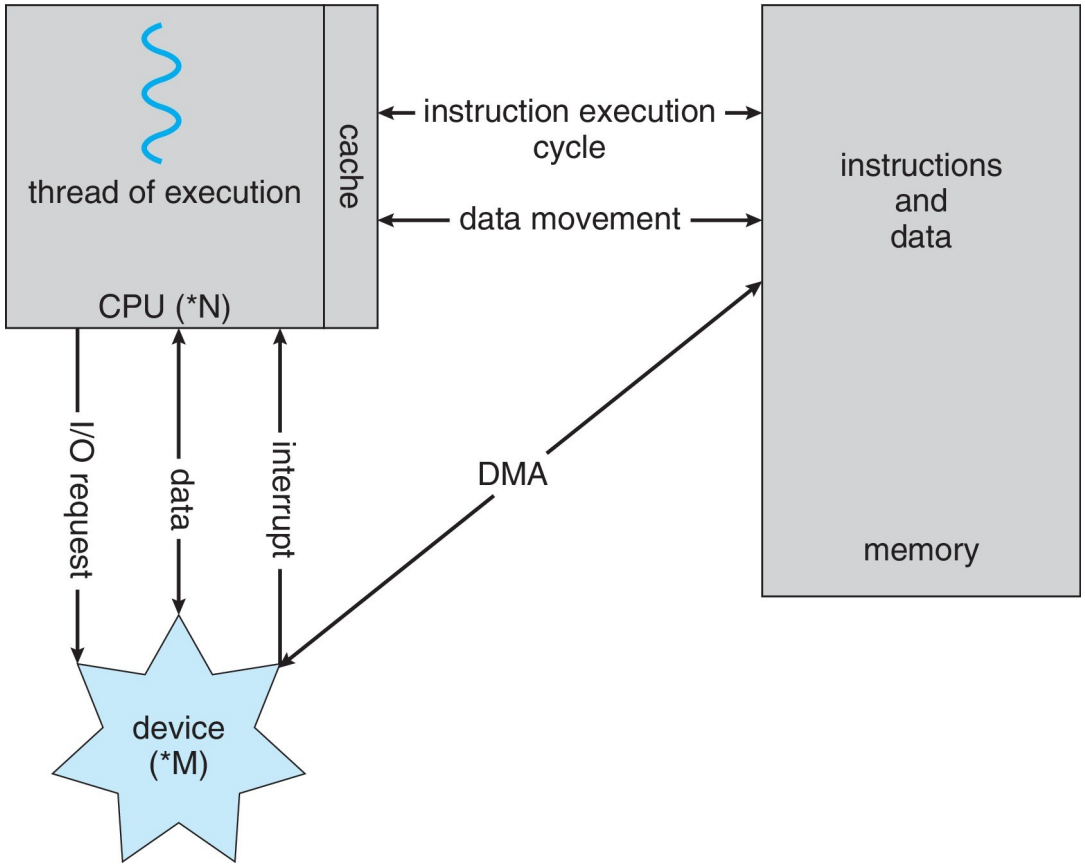
- **Caching** – copying information into faster storage system; main memory can be viewed as a cache for secondary storage

- **Device Driver** for each device controller to manage I/O
  - Provides uniform interface between controller and kernel

Storage-Device Hierarchy



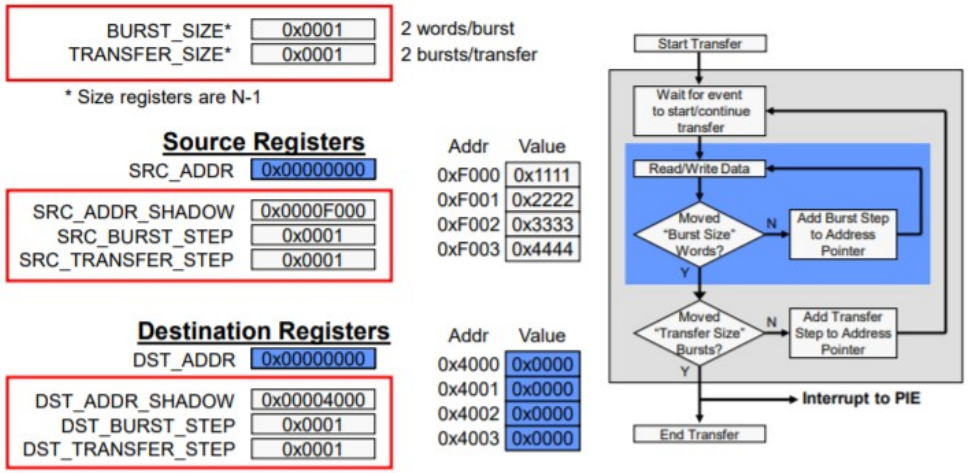
How a Modern Computer Works



A von Neumann architecture

Direct Memory Access Structure

- Used for high-speed I/O devices able to transmit information at close to memory speeds
- Device controller transfers blocks of data from buffer storage **directly** to main memory without CPU intervention
  - CPU can keep running processes while the buffer is transferred concurrently
- Only one interrupt is generated per block, rather than the one interrupt per byte

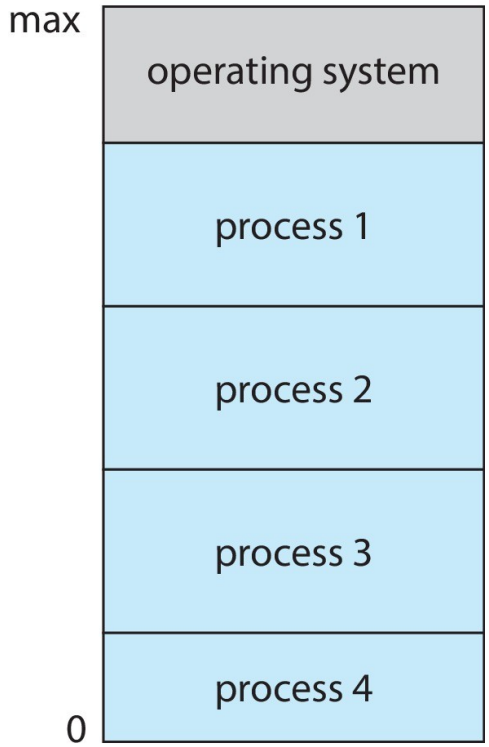


Source: TI C28x Academy

Multitasking (Timesharing)

- Nowadays OSs allow **running several tasks concurrently**. The CPU switches jobs so frequently that users can interact with each job without noticing the switching – allows **interactive** computing
  - **Response time** should be small (depends on application, but usually of the order of ~10's to 100's of ms)
  - Each user can have a potentially unlimited number of programs executing in memory (**process)**
  - If several jobs are ready to run at the same time, the OS selects which one executes (**process scheduling)**
  - If processes don't fit in memory, **swapping** moves parts of them (data and code) in and out, while they run
    - **Virtual memory** allows execution of processes not completely in memory

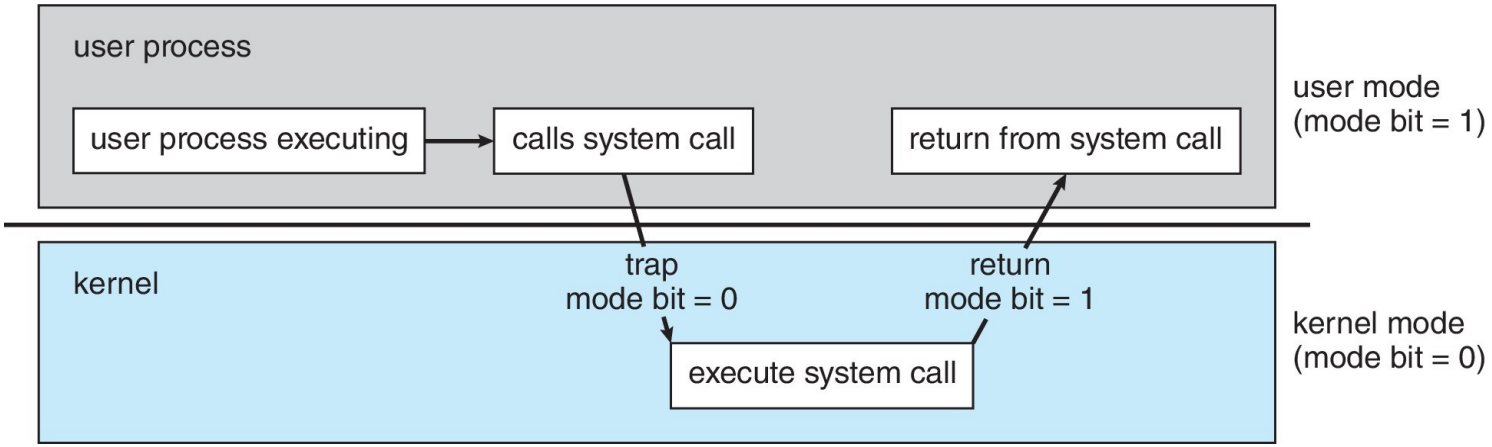
Memory Layout for Multiprogrammed System



Dual-mode Operation

- **Dual-mode** operation allows the OS to protect itself and other system components
  - **User mode** and **kernel mode**
- **Mode bit** provided by hardware
  - Provides ability to distinguish when system is running user code or kernel code.
  - When a user is running → mode bit is “user”
  - When kernel code is executing → mode bit is “kernel”
- How do we guarantee that user does not explicitly set the mode bit to “kernel”?
  - System call changes mode to kernel, return from call resets it to user
- Some instructions designated as **privileged**, only executable in kernel mode

Transition from User to Kernel Mode



Process Management

- **A process is a program in execution.**

  It is a unit of work within the system. A program is a ***passive entity;*** A process is an ***active entity***.

- A Process needs resources to accomplish its task
  - CPU, memory, I/O, files
  - Initialization data
- Process termination requires reclaiming of any reusable resources
- A single-threaded process has one **program counter** specifying location of next instruction to execute
  - Process executes instructions sequentially, one at a time, until completion
- Multi-threaded process has one program counter per thread
- Typically a system has many processes, eventually from different users, and operating system functions (aka system processes), all running concurrently on one or more CPUs
  - Concurrency is achieved by multiplexing the CPUs among the processes / threads

Process Management Activities

The operating system is responsible for the following activities in connection with process management:

- **Creating** and **deleting** both user and system processes
- **Suspending** and **resuming** processes
- Providing mechanisms for process **synchronization**
- Providing mechanisms for process **communication**
- Providing mechanisms for **deadlock handling**

Memory Management

- To execute a program all (or part) of the instructions must be in memory
  - The “LIVE parts” (i.e. the instructions and variables being processed at any given moment) must be in main memory
- Memory management determines what is in memory and when
  - The objective is optimizing CPU utilization and computer response to users
- Memory management activities
  - Keeping track of which parts of memory are currently being used and by which processes/threads
  - Deciding which processes (or parts of it) and data to move into and out of memory
  - Allocating and deallocating memory space as needed

File-system Management

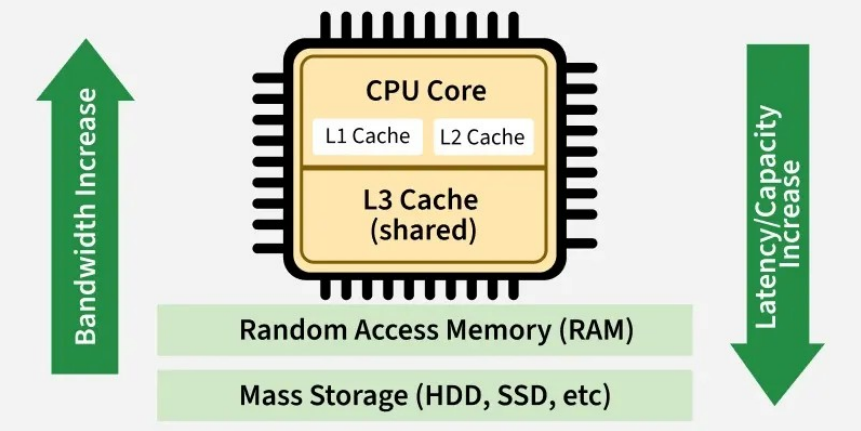
- OS provides uniform, logical view of information storage
  - Abstracts physical properties to logical storage unit - **file**
  - Each medium is controlled by device (i.e., disk drive, USB drive)
    - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
  - Files usually organized into directories (or folders)
  - Access control on most systems to determine who can access what
  - OS activities include:
    - **Creating** and **deleting files** and **directories**
    - Primitives to **manipulate** files and directories
    - Mapping files onto secondary storage
    - Backup files onto stable (non-volatile) storage media

Mass-Storage Management

- Usually disks used to store data that does not fit in main memory or data that must be kept for a “long” period of time
- Proper management is of central importance
- Entire speed of computer operation hinges on disk subsystem and its management algorithms
- OS activities:
  - Mounting and unmounting
  - Free-space management
  - Storage allocation
  - Disk scheduling
  - Partitioning
  - Protection

Caching

- Important principle, performed at many levels in a computer
  - In hardware, operating system, software



- Data in use copied from slower to faster storage temporarily

- Faster storage (cache) checked first to determine if data is there
  - If it is, information used directly from the cache (fast)
  - If not, data is copied to cache and used from there (slow)

- Cache is orders of magnitude smaller than storage being cached
  - Cache management is an important design problem
  - Cache size and replacement policy have a significant performance impact

Characteristics of Various Types of Storage

Level	1	2	3	4	5
Name	registers	cache (L1/L2/L3)	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB (per core)	< 128 MB	< 256 GB	< 8 TB	< 30 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip CMOS SRAM	DDR5 DRAM / HBM3E	3D NAND flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	1 - 15	75 - 100	~ 10,000	5,000,000 - 10,000,000
Bandwidth (MB/sec)	100,000 - 1,000,000+	50,000 - 1,000,000	50,000 - 1,200,000	7,000 - 14,900	150 - 300
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	solid-state disk	magnetic disk	tape / cloud storage

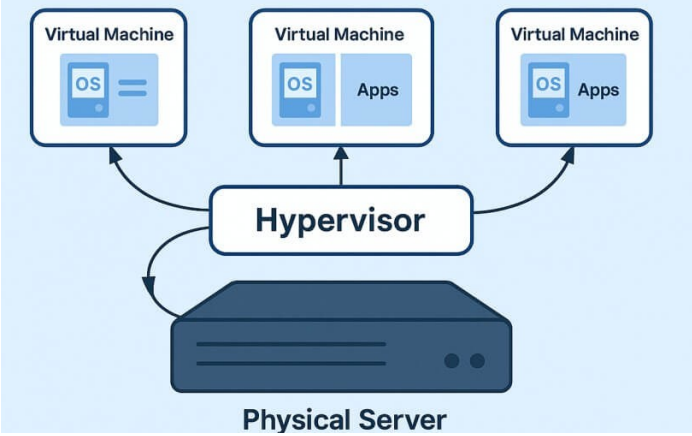
Note: based on Fig. 1.14 of the main book, but updated to reflect typical characteristics of current market products as of 2026

Protection and Security

- **Protection mechanism** – any mechanism for controlling access of processes or users to resources managed by the OS
- **Security** – defense of the system against internal and external attacks
  - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
  - User identities (**user IDs**, security IDs) include name and associated number, one per user
  - User ID then associated with all files and processes of that user to determine access control
  - Group identifier (**group ID**) allows sets of users to be defined and grant common access control rights
  - **Privilege escalation** allows user to change to effective ID with more rights

Virtualization

- Allows operating systems to run applications within other OSes
  - Vast and growing industry
- **Emulation** used when source CPU type different from target type (i.e. PowerPC to Intel x86)
  - Generally slowest method
  - When computer language not compiled to native code – **Interpretation**
- **Virtualization** – OS natively compiled for the CPU, running **guest** OSes also natively compiled
  - E.g. VirtualBox or VMWare running Windows or Linux guests, each running applications, all on native Linux **host** OS
  - **VMM** (virtual machine Manager) provides virtualization services



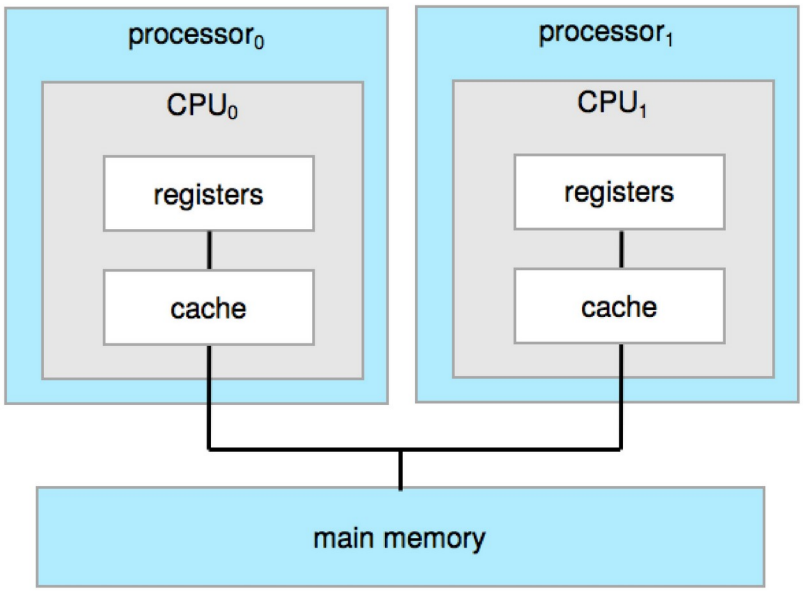
Computer System Architecture

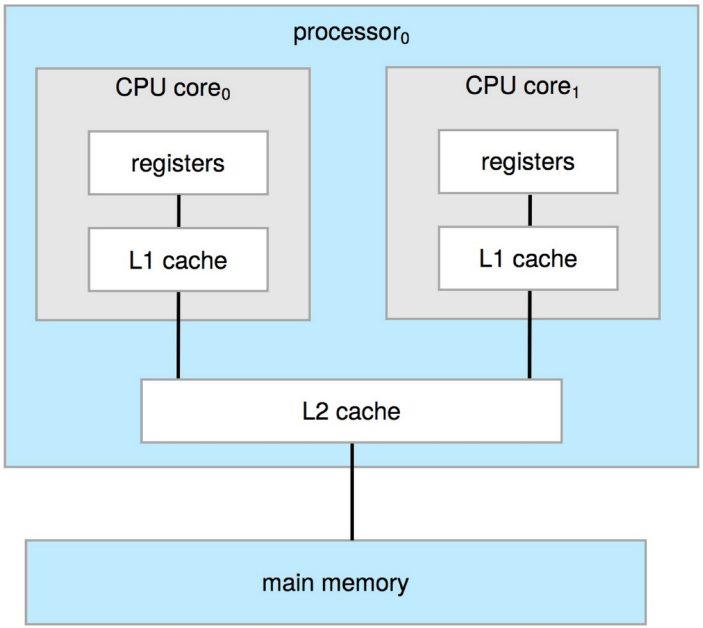
Computer-System Architecture

- Traditionally most systems used a single general-purpose processor, eventually complemented by special-purpose (co-)processors as well
- Eventually **multiprocessor** systems growth in use and importance, even in the embedded domain
  - Also known as **parallel systems**, **tightly-coupled systems**
  - Advantages include:
    1. **Increased throughput**
    2. **Economy of scale**
    3. **Increased reliability** – graceful degradation or fault tolerance
  - Two types:
    1. **Asymmetric Multiprocessing** – different processors are treated differently: e.g. a system can allow only one CPU to execute OS code or allow only one CPU to perform I/O operations.
    2. **Symmetric Multiprocessing** – all processors share main memory, have full access to all I/O devices and are managed by a single OS that treats them equally (no processor is reserved for special purposes)

Symmetric Multiprocessing Architecture

- Multi-processor and **multicore**
- One or more CPUs, each one with one or more cores





Computer System Environments

General-Purpose Operating Systems (GPOS)

- A GPOS is designed to run many kinds of programs for many users. Its main characteristics are:
  - **Multitasking & multiuser**: runs several programs and supports multiple users at once.
  - **Hardware abstraction**: hides hardware details and provides standard interfaces.
  - **Resource management**: allocates CPU, memory, storage, and I/O efficiently.
  - **File system**: organizes and secures persistent data.
  - **Networking**: enables communication and internet access.
  - **Security**: protects users and processes from each other.
  - **Portability/extensibility**: works on different HW and supports new features.
  - **Interactive & batch use**: supports both direct interaction (CLI/GUI) and scheduled jobs.
- Examples: Linux, Windows, macOS

Real-Time Operating Systems

- Embedded systems are the most prevalent form of computers, although most people don't realize it (why?)
  - Characteristics vary considerable
  - Often have a special/limited purpose
  - Traditionally **Direct Programming**, but the **use of OSs** is quickly become prevalent (why?)
  - Usually are **resource-limited** (10's to 100's of MHz, kB to MB of RAM, ...)
- Real-time OSs (for embedded systems or not) support SW development with well-defined fixed time constraints
  - Processing ***must*** be done within time constraints
  - Correct operation only if:
    - Result **logically correct and produced in time**!



- Examples of RTOS:
  - Zephyr, FreeRTOS, Xenomai
- Some GPOS support (limited) RT. E.g.:
  - Linux RT services, Windows 10 IoT Enterprise Soft Real-time services

Free and Open-Source Operating Systems

- Operating systems made available in source-code format rather than just binary **closed-source** and **proprietary**
- Counter to the **copy protection** and **Digital Rights Management** (**DRM**) movement
- Started by **Free Software Foundation** (**FSF**), which has “copyleft” **GNU Public License** (**GPL**)
  - Free software and open-source software are two different ideas championed by different groups of people
    - **https://www.gnu.org/philosophy/open-source-misses-the-point.en.html**
- Examples include **GNU/Linux** and **BSD UNIX** (including core of **Mac OS X**), and many more
- Can use VMM like VMware Player (Free on Windows), Virtualbox (open source and free on many platforms - http://www.virtualbox.com)
  - Use to run guest operating systems for exploration



- Questions?

Acknowledgment:

This presentation is based on the companion materials provided with the book

Operating System Concepts – 10h Edition, by Silberschatz, Galvin and Gagne